Labo 2 Geavanceerde computerarchitectuur

Gust Brouckaert - Tim Depickere

2024-11-09

1 Introductie

In dit tweede labo leerden we over race conditions. We moesten max detection, waarbij je een array doorloopt en het grootste element vindt, toepassen op drie verschillende manieren:

- Een geneste for-loop op de CPU.
- Een GPU-kernel met behulp van atomaire operaties.
- Een GPU-kernel met behulp van reductie.

2 Nested for loop op de CPU

Hierbij overlopen we alle elementen in de array en slaan deze waarde op in een integer `max` als deze groter is dan de huidige maximale waarde.

```
void maxDetectionCPU(int *array, int size, int* max) {
    for(int i = 0; i < size; i++) {
        if(array[i] > *max){
            *max = array[i];
        }
    }
}
```

3 GPU kernel mbv atomische operatie

Het doel van atomaire operaties is ervoor te zorgen dat niet meerdere threads tegelijkertijd de maximale waarde aanpassen, wat zou leiden tot een data-race. Eerst berekenen we de `idx` om te zorgen dat elke thread werkt met de correcte index, ook als er meerdere blocks in de grid zijn. Vervolgens gebruiken we de functie `atomicMax` uit de CUDA-bibliotheek. Het principe van de maximumwaarde vinden blijft hetzelfde als bij de geneste for-loop op de CPU.

```
__global__ void maxDetectionGPUKernel(int *array, int* max) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    atomicMax(max, array[idx]);
}
```

4 GPU kernel mbv reductie

Bij de reductiemethode gebruiken we een algoritme dat telkens de maximale waarden paarsgewijs vergelijkt en samenvat. Elke thread vergelijkt zijn waarde met een andere waarde in de array en houdt de hoogste bij. Door na elke iteratie de functie `__syncthreads()` toe te passen, zorgen we ervoor dat alle threads gesynchroniseerd blijven, zodat er geen fouten optreden door gelijktijdige toegang tot gedeelde data. Wij doen dit nu in de originele array. Je kunt ook een gedeelde array (`sharedData`) aanmaken om dit op te slaan. Dit hebben wij niet gedaan aangezien dit niet werkte bij ons, de reden hiervoor weten we momenteel niet. Het voordeel van het gebruik van Shared memory is dat deze memory als een soort cache werkt voor alle threads uit een block. Hierdoor zal het gebruik hiervan zorgen voor een prestatie verbetering aangezien de shared memory minder lang zal duren om te gebruiken.

```
__device__ int maxNumber(int a, int b) {
    return (a > b) ? a : b;
}

__global__ void maxDetectionGPUReduction(int *array, int size) {
    int idx = threadIdx.x;

    //extern __shared__ int sharedData[];
    //sharedData[idx] = array[idx];
    //__syncthreads();

    for (int i = 1; i < blockDim.x; i *= 2) {
        int index = 2 * i * idx;
        if (index < blockDim.x) {
            array[index] = maxNumber(array[index], array[index + i]);
        }
        __syncthreads();
    }

    /*if (idx == 0) {
        array[0] = sharedData[0];
    }*/
}
```

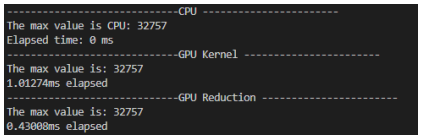
5 Extra questions

5.1 Is het resultaat correct?

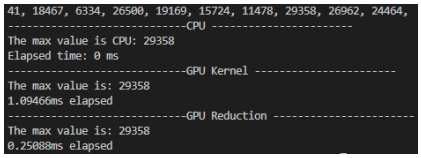
Het resultaat is bij alle 3 gelijk en werd ook handmatig gecontroleerd voor de kleinere arrays. Hieruit concluderen wij dat het resultaat correct is.

5.2 Hoe verandert de tijd in functie van input groote?

Op onderstaande afbeeldingen hebben we de verwerkingstijd voor twee arrays van verschillende groottes vergeleken. Figuur (a) toont een array met grootte 1024, en figuur (b) toont een array van grootte 10. Het grootste verschil in verwerkingstijd komt voor bij de GPU-reductie. Hier is de verwerkingstijd bijna verdubbeld wanneer de arraygrootte 100 keer groter wordt. De GPU-kernel met atomische operaties duurt wel nog steeds langer, maar is slechts met minder dan 10% gestegen. De CPU-tijd laat geen significante stijging zien en blijft rond de 0 ms. Bij metingen in microseconden kwam dit neer op ongeveer 500 μs.



(a) Tijdswaardes voor array[1024]



(b) Tijdswaardes voor array[10]

5.3 Wat gebeurt er als input groter is dan 2048?

Wanneer de input groter is dan 2048 elementen, is het niet mogelijk om alle elementen binnen één block te verwerken omdat er per block maximaal 1024 threads beschikbaar zijn. Bij grotere input moeten de elementen verdeeld worden over meerdere blocks. Hierbij moet er synchronisaties plaatsvinden tussen de blocks om te zorgen dat de resultaten correct worden samengevoegd.

5.4 Overweeg het aanroepen van de kernel met veel minder threads dan er data-elementen zijn.

Wanneer we de kernel aanroepen met minder threads dan het aantal data-elementen, kunnen we gebruik maken van een techniek genaamd "striding". Elke thread verwerkt dan meerdere elementen door het totale aantal data-elementen op te delen over de beschikbare threads. Dit kan door elke thread in een loop te laten werken, waarbij de loop telkens met een stride ter grootte van het totaal aantal threads de volgende elementen doorloopt. Dit zorgt ervoor dat elke thread alle toegewezen elementen in de array verwerkt.

5.5 Hoe zou je een 2D array gebruiken.

De aanpak voor een 2D-array kan gebaseerd zijn op dezelfde principes als voor een 1D-array. Elke rij kan worden doorlopen waarbij de grootste waarde in de eerste positie van de rij (bijvoorbeeld op rij[0]) wordt geplaatst. Vervolgens kan de eerste kolom (kolom[0]) doorlopen worden, waarbij de grootste waarde in het element array[0][0] wordt geplaatst.